

Analyse de Sentiments sur les Critiques de Films

Zakarya Elmimouni

Yassine Boukhateb

Mai 2025

1 Introduction

L'analyse de sentiments, ou "opinion mining", c'est le fait d'essayer de comprendre ce que pense ou ressent une personne à partir de ce qu'elle écrit. Par exemple, on peut vouloir savoir si une critique de film est positive ou négative. C'est un domaine du traitement automatique du langage naturel (NLP), qui est de plus en plus utilisé aujourd'hui, surtout avec la quantité énorme de textes publiés chaque jour sur les réseaux sociaux, les forums ou les sites d'avis.

Dans ce projet, ayant pour point de départ la lecture du papier de Maas et al. (2011) recommandé par l'enseignant, notre but est de créer des modèles capables de détecter automatiquement le sentiment exprimé dans des critiques de films. Pour ça, on teste plusieurs approches : des méthodes classiques comme la régression logistique, mais aussi des modèles plus récents et puissants comme BERT ou DeBERTa. On s'intéresse aussi à différentes façons de représenter le texte, comme TF-IDF ou les embeddings de mots. À la fin, on compare les résultats pour voir quelles méthodes fonctionnent le mieux, notamment la méthode de Bert, la plus récente, et ce que chaque type de modèle apporte.

2 Revue de la littérature

L'analyse de sentiments a beaucoup évolué ces dernières années, en grande partie grâce aux progrès dans le traitement du langage naturel (NLP) et le deep learning. Au départ, les méthodes étaient plutôt simples : on représentait le texte comme une liste de mots (avec des techniques comme *Bag-of-Words* ou TF-IDF), et on utilisait des modèles classiques comme la régression logistique, le Naive Bayes ou les SVM (Pang and Lee, 2002).

Puis sont arrivés les *word embeddings*, comme Word2Vec (Mikolov et al., 2013) ou GloVe (Pennington et al., 2014), qui permettent de représenter les mots avec des vecteurs qui capturent leur sens. Ces représentations ont vraiment amélioré les performances des modèles.

Ensuite, les réseaux de neurones comme les RNN, LSTM ou GRU ont été très utilisés, avant d'être dépassés par les modèles à base de *Transformers*, comme BERT (Devlin et al., 2019). Ces modèles sont capables de comprendre le contexte d'un mot dans une phrase, et offrent des résultats impressionnants une fois ajustés sur des tâches spécifiques comme l'analyse de sentiments.

Des variantes plus légères ou spécialisées sont aussi apparues, comme DistilBERT, RoBERTa, ou encore des modèles spécialement entraînés pour analyser les sentiments.

Enfin, les résultats dépendent aussi beaucoup du jeu de données utilisé. Les plus connus dans ce domaine sont IMDB (Maas et al., 2011), SST-2 (Socher et al., 2013), ou les avis clients d'Amazon.

3 Jeu de données

Le jeu de données utilisé dans ce projet est le **IMDB Large Movie Review Dataset** Maas et al. (2011), un corpus très populaire pour l'analyse de sentiments en anglais, surtout dans le domaine des critiques de films.

3.1 Description

Ce dataset contient **50 000 critiques de films**, réparties équitablement entre deux classes :

- **Critiques positives** (label = 1)
- **Critiques négatives** (label = 0)

Les données sont également divisées en deux parties :

- **Train** : 25 000 critiques
- **Test** : 25 000 critiques

Chaque critique est une simple chaîne de texte, déjà nettoyée (pas de balises HTML, pas de doublons), avec en moyenne une phrase par fichier.

3.2 Structure du dossier

Une fois téléchargé, le dataset est organisé ainsi :

```
data/  
  train/  
    pos/      ← 12 500 critiques positives  
    neg/      ← 12 500 critiques négatives  
  test/  
    pos/      ← 12 500 critiques positives  
    neg/      ← 12 500 critiques négatives
```

Chaque fichier texte (.txt) contient une critique encodée en UTF-8.

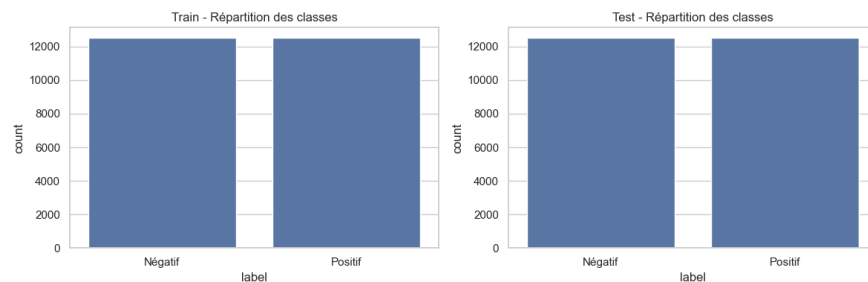


FIGURE 1 – Répartition des classes dans le dataset

3.3 Exemples

Voici deux exemples de critiques extraites du dataset :

- **Critique positive** :
“This movie was fantastic! The performances were outstanding, and the story was both emotional and inspiring. Highly recommended!”
- **Critique négative** :
“What a waste of time. The plot made no sense, the acting was wooden, and I couldn’t wait for it to end. Absolutely terrible.”

4 Analyse exploratoire des données

Avant d’entraîner les modèles, une première analyse a été réalisée pour comprendre la structure des textes, notamment en étudiant la longueur des critiques (en nombre de mots), aussi bien pour l’entraînement que pour le test.

4.1 Distribution des longueurs de critiques

Chaque critique a été tokenisée en séparant les mots par des espaces. On a ensuite tracé la distribution des longueurs pour les deux ensembles. Le résultat montre des distributions très proches, centrées autour de 200 à 300 mots.

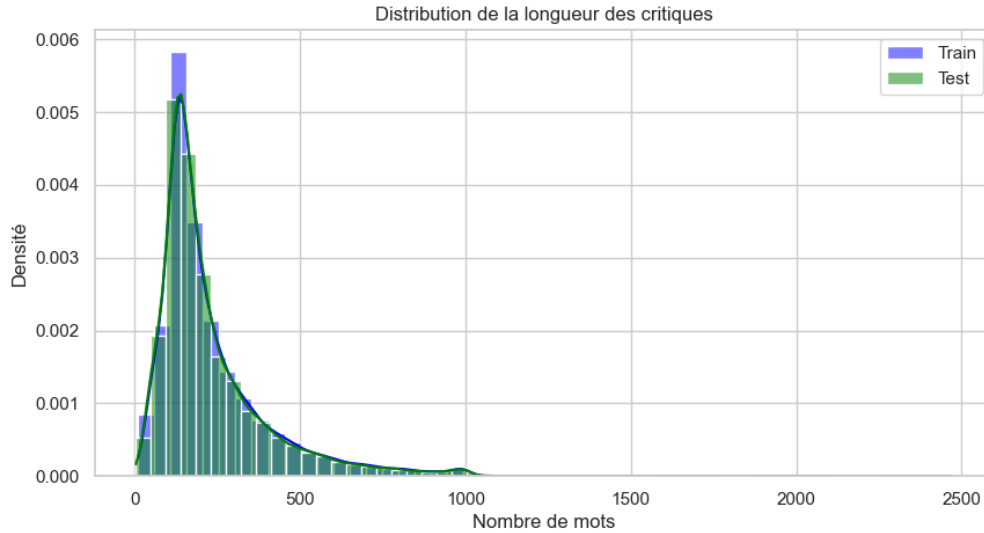


FIGURE 2 – Distribution des longueurs de critiques dans les ensembles **train** et **test**

Cette similarité confirme que les deux ensembles sont bien équilibrés et comparables, ce qui est essentiel pour un bon entraînement et une évaluation fiable des modèles.

4.2 Nuages de mots par polarité

Nous avons généré des *word clouds* afin de visualiser les mots les plus fréquents dans les critiques à **sentiment positif** et **sentiment négatif**.



FIGURE 3 – Nuages de mots pour les critiques positives (gauche) et négatives (droite)

Le nuage de mots pour les critiques positives met en évidence des termes comme *good*, *best*, *interesting*, traduisant une appréciation forte. À l'inverse, les critiques négatives sont dominées par des mots tels que *bad*, *awful*, *waste*, traduisant une insatisfaction.

Ces visualisations qualitatives confirment que les critiques sont bien différenciées lexicalement selon leur polarité, ce qui renforce la pertinence d'une approche basée sur le langage.

4.3 Analyse des bigrammes

Afin d'identifier les associations de mots les plus fréquentes selon le sentiment exprimé, nous avons extrait les **20 bigrammes les plus fréquents** dans les critiques *positives* et *négatives*.

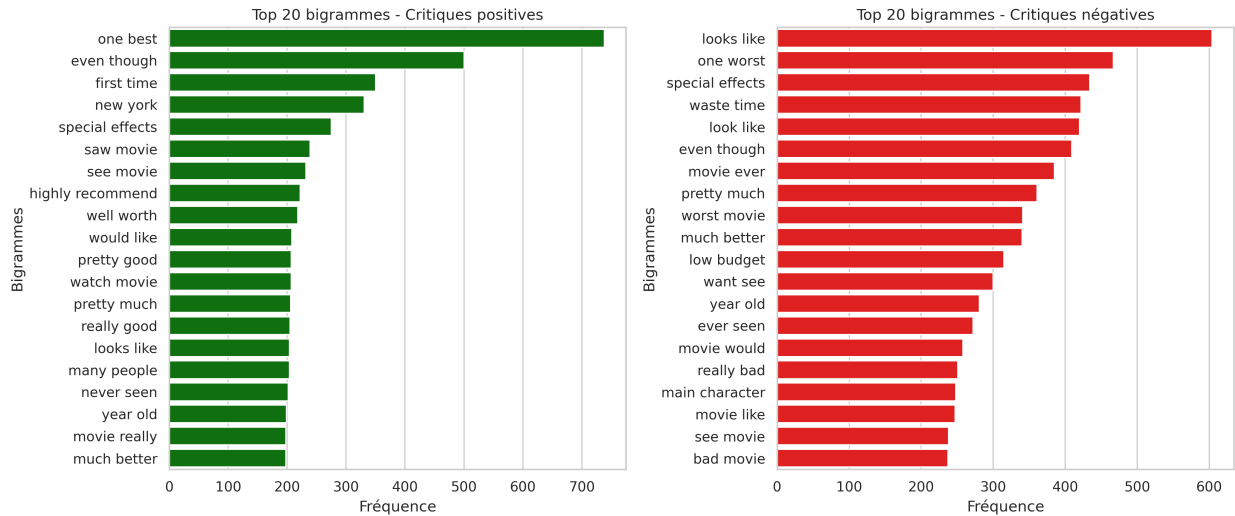


FIGURE 4 – Top 20 des bigrammes les plus fréquents dans les critiques positives (à gauche) et négatives (à droite)

Comme le montre la figure 4, les critiques positives contiennent des expressions valorisantes telles que “*one best*”, “*highly recommend*”, “*really good*” ou encore “*well worth*”, traduisant une forte appréciation du film.

À l’inverse, les critiques négatives sont marquées par des combinaisons comme “*one worst*”, “*waste time*”, “*really bad*”, ou encore “*low budget*”, qui soulignent la déception ou la frustration du spectateur.

L’apparition de certains bigrammes dans les deux classes (“*looks like*”, “*special effects*”, etc.) suggère qu’un même thème peut être abordé sous un angle positif ou négatif, selon le contexte.

5 Implémentation de modèles

5.1 Modèle de base avec TF-IDF

Comme point de départ, nous avons implémenté un modèle simple combinant une vectorisation TF-IDF des critiques avec un classifieur *Naive Bayes multinomial*. Ce modèle repose sur l’hypothèse d’indépendance conditionnelle des mots, ce qui en fait une approche naïve mais rapide à mettre en œuvre. Bien qu’efficace sur des textes courts et bien séparés, ce type de modèle atteint rapidement ses limites pour capturer les relations contextuelles ou sémantiques plus subtiles entre les mots. Il a ainsi servi de référence initiale, avant de passer à des approches plus complexes et performantes. Ce modèle a tout de même obtenu une **accuracy de 84% sur l’ensemble de test**, ce qui souligne l’intérêt d’un prétraitement efficace même dans un cadre simple.

5.2 Version simplifiée du modèle de l’article

Avant de mettre en œuvre le modèle complet comme décrit dans l’article de Maas et al. (Maas et al., 2011), nous avons d’abord choisi de créer une version simplifiée afin d’évaluer les performances et de tester la faisabilité du projet. Nous avons utilisé des techniques classiques de machine learning pour commencer, avec une vectorisation de texte basique, avant de passer à des approches plus avancées.

5.2.1 Vectorisation des textes

Afin d'alimenter nos modèles avec des données numériques exploitables, nous avons vectorisé les critiques textuelles en utilisant un `CountVectorizer` de `SCIKIT-LEARN`. Ce vecteur représente chaque critique par une série de caractéristiques numériques issues de ses mots.

Nous avons limité notre vocabulaire aux **5 000 mots les plus fréquents** dans l'ensemble de données, et avons exclu les mots communs (*stopwords*) en anglais. La représentation choisie est binaire, ce qui signifie que chaque critique est transformée en un vecteur indiquant la présence ou l'absence de chaque mot du vocabulaire :

- Chaque critique est transformée en un vecteur binaire de dimension 5 000.
- Ainsi, les matrices X_{train} et X_{test} résultantes sont de taille `[n_documents × 5000]`.
- Les labels associés aux critiques sont binaires : 0 pour une critique négative, 1 pour une critique positive.

Cette approche simplifiée a pour objectif de créer une première base de travail pour des tests rapides et une évaluation préliminaire.

5.3 Apprentissage sémantique non supervisé

Dans cette section, nous implémentons le modèle log-linéaire comme décrit dans la section 3.1 du papier de Maas et al. (Maas et al., 2011) pour modéliser la distribution des mots dans un document en fonction de sa représentation latente.

5.3.1 Modélisation de la log-vraisemblance

Pour modéliser cette distribution, nous avons défini une fonction de vraisemblance conditionnelle pour chaque mot w dans un document d :

$$p(w \mid \theta_d) = \frac{\exp(\theta_d^\top \phi_w + b_w)}{\sum_{w' \in V} \exp(\theta_d^\top \phi_{w'} + b_{w'})}$$

Où :

- ϕ_w est le vecteur d'embedding associé au mot w du vocabulaire.
- b_w est un biais pour chaque mot.
- θ_d est un vecteur latent propre à chaque document, qui est optimisé pendant l'entraînement par rétropropagation.

Nous maximisons la log-vraisemblance des mots observés dans chaque document, qui peut être exprimée comme suit :

$$\log p(d) = \sum_{w \in d} \log p(w \mid \theta_d)$$

Cette équation représente la probabilité de la distribution des mots donnée la représentation θ_d d'un document. L'objectif est de minimiser la *négative log-vraisemblance* (NLL) en utilisant un optimiseur comme `Adam` pour apprendre les paramètres du modèle sur plusieurs époques.

5.3.2 Entraînement du modèle

Le modèle est entraîné à l'aide de l'optimiseur `Adam` et la fonction de perte est optimisée sur plusieurs époques. Chaque époque consiste à effectuer une passe à travers l'ensemble d'entraînement, ajustant les poids du modèle à chaque itération. Ce processus permet au modèle d'affiner ses prédictions en fonction des données d'entraînement.

- L'optimiseur `Adam` est utilisé pour ajuster les poids.
- La fonction de perte `BCEWithLogitsLoss` est utilisée pour la classification binaire.
- Le processus d'entraînement est effectué sur plusieurs époques pour optimiser les paramètres.

5.3.3 Classification supervisée à partir des vecteurs appris

Une fois les vecteurs de mots ϕ_w appris, nous avons projeté chaque document dans l'espace latent via la formule suivante :

$$\theta_d = \frac{1}{|d|} \sum_{w \in d} \phi_w$$

Ces représentations θ_d ont ensuite été utilisées comme entrées d'un modèle de **régression logistique** pour prédire le sentiment des critiques. Nous avons optimisé l'hyperparamètre de régularisation C via une validation croisée avec `GridSearchCV`, et évalué les performances obtenues sur l'ensemble d'entraînement et de test.

- **Accuracy entraînement** : 69%
- **Accuracy test** : 69.7%

5.3.4 Exploration d'autres classifieurs

En complément de la régression logistique, nous avons testé deux autres modèles classiques de classification supervisée sur les représentations θ_d des documents :

- **Random Forest**
- **XGBoost**

Nous avons utilisé une recherche d'hyperparamètres via `GridSearchCV` pour optimiser les configurations de ces modèles. Les performances obtenues étaient proches de celles de la régression logistique, avec une légère amélioration sur le jeu d'entraînement pour XGBoost, mais sans gain significatif sur le test.

Cette comparaison montre que la qualité des vecteurs ϕ_w appris via ce modèle n'est pas trop forte suite aux résultats obtenus.

5.4 Classification Zero-Shot avec DistilBERT

Nous avons initialement envisagé de **fine-tuner le modèle BERT** sur notre jeu de données. Cependant, cette opération s'est révélée trop coûteuse en termes de mémoire et de ressources de calcul, notamment en environnement CPU ou avec des contraintes de RAM. En raison de ces limitations matérielles, nous avons opté pour une approche alternative plus légère : la **classification zero-shot à l'aide d'un modèle BERT pré-entraîné**.

5.4.1 Historique du modèle BERT

Le modèle BERT (*Bidirectional Encoder Representations from Transformers*), introduit par Devlin et al. en 2018 (Devlin et al., 2019), a marqué une rupture importante par rapport aux approches précédentes, comme celles utilisées dans l'article de Maas et al. (Maas et al., 2011). Contrairement à des modèles plus simples basés sur des représentations statiques des mots (comme les embeddings Word2Vec ou GloVe), BERT utilise des représentations contextuelles, où chaque mot est représenté différemment selon son contexte dans la phrase. Cela lui permet de capturer des relations complexes entre les mots et améliore considérablement les performances sur des tâches variées de traitement du langage naturel, dont l'analyse de sentiments.

5.4.2 Principe de l'approche

L'approche BERT repose sur un modèle de type Transformer, qui est bidirectionnel et pré-entraîné sur un large corpus de texte avant d'être fine-tuné sur des tâches spécifiques, comme la classification des sentiments. Cette capacité à apprendre de manière contextuelle et à être fine-tuné rapidement sur des tâches spécifiques en fait l'un des modèles les plus performants et polyvalents pour de nombreuses applications en NLP. En raison de ses excellentes performances sur des benchmarks standards, BERT est désormais considéré comme un modèle de référence *state-of-the-art* dans le domaine de l'analyse de sentiments, surpassant les modèles traditionnels et même ceux qui combinent des techniques d'apprentissage supervisé et non supervisé comme celles proposées par Maas et al.

Dans ce projet, nous avons utilisé le modèle `distilbert-base-uncased-finetuned-sst-2-english`, proposé par Hugging Face, qui est une version compacte de BERT fine-tunée sur le corpus SST-2 (Stanford Sentiment Treebank). Ce modèle est capable de prédire directement la polarité (positive ou négative) d'un texte, sans nécessiter d'entraînement supplémentaire.

L'inférence est réalisée via la fonction `pipeline("text-classification")`, ce qui permet d'obtenir rapidement une prédiction de sentiment pour chaque critique. Cette approche, bien qu'allégée, permet de bénéficier des avantages de BERT tout en réduisant les contraintes liées à l'entraînement du modèle, tout en maintenant des performances proches de l'état de l'art.

5.4.3 Évaluation sur notre jeu de test

Nous avons appliqué cette approche sur notre ensemble de test (non vu par le modèle), en lui fournissant les critiques brutes. Le modèle prédit un label `POSITIVE` ou `NEGATIVE`, que nous avons ensuite comparé aux vrais labels du dataset IMDB.

— **Test accuracy obtenue : 89%**

Ce résultat montre que les modèles pré-entraînés peuvent atteindre des performances très satisfaisantes sans étape de fine-tuning, ce qui en fait une solution très adaptée pour des contextes à ressources limitées.

5.5 Comparaison des performances des modèles

Le tableau ci-dessous présente les différentes approches que nous avons testées dans ce projet, avec leur précision (*accuracy*) sur l'ensemble de test. Ce tableau permet de mieux comprendre l'impact de chaque méthode sur les résultats de la classification de sentiments.

Modèle	Accuracy Test (%)
TF-IDF + Naive Bayes	84.0
Modèle simplifié du papier ϕ_w	87.3
Régression logistique sur les vecteurs ϕ_w	69.7
Random Forest sur les vecteurs ϕ_w	67.6
XGBoost sur les vecteurs ϕ_w	68.5
Zero-shot (DistilBERT pré-entraîné, pipeline)	89.0

TABLE 1 – Comparaison des performances sur l'ensemble de test

5.5.1 Analyse des résultats

D'après les résultats obtenus, nous pouvons observer quelques points clés concernant les différentes approches :

- **Modèle simplifié du papier (ϕ_w)** : Ce modèle atteint une précision de 87.3%, ce qui est relativement élevé. Il combine un apprentissage sémantique non supervisé avec une régression logistique, ce qui semble bien capturer la structure sémantique des critiques. Cela montre que l'utilisation d'embeddings de mots peut être efficace pour ce type de tâche, même sans l'ajout de modèles plus complexes.
- **Modèles classiques (Régression logistique, Random Forest, XGBoost)** : Les modèles classiques appliqués sur les vecteurs ϕ_w (régression logistique, Random Forest et XGBoost) ont montré des performances variées. La régression logistique a donné un résultat de 69.7%, ce qui est déjà raisonnable mais inférieur à celui du modèle du papier. Les performances des autres modèles étaient encore plus faibles, notamment pour Random Forest (67.6%) et XGBoost (68.5%), ce qui suggère que ces méthodes ne tirent pas pleinement parti des vecteurs de mots appris, probablement en raison de la complexité de la tâche.
- **Zero-shot avec DistilBERT** : L'approche la plus impressionnante est l'utilisation de **DistilBERT**, un modèle pré-entraîné dans un cadre zero-shot. Cette méthode a permis d'atteindre une précision de 89% sur l'ensemble de test, ce qui dépasse les autres modèles. Cette performance démontre la puissance des modèles pré-entraînés comme DistilBERT, qui peuvent réaliser des tâches complexes de classification de manière efficace sans nécessiter de fine-tuning, surtout dans des environnements à

ressources limitées. Il est également important de noter que cette méthode est très rapide à mettre en œuvre et ne nécessite pas un long temps d’entraînement.

5.5.2 Points clés à retenir

- L’approche utilisant les embeddings de mots pré-appris, comme dans le modèle simplifié, s’est avérée très efficace.
- Les modèles classiques comme Random Forest et XGBoost ont montré des résultats plus faibles, probablement à cause de leur incapacité à tirer pleinement parti de la richesse sémantique des embeddings.
- **DistilBERT**, un modèle pré-entraîné, a montré des performances exceptionnelles en mode zero-shot, sans nécessiter d’entraînement supplémentaire.

Répartition des tâches

La répartition des tâches pour ce projet a été la suivante :

- **Zakarya** a pris en charge l’essentiel du code et de l’implémentation de la méthodologie du papier. Il a réalisé la modélisation de l’apprentissage sémantique non supervisé et la classification supervisée basée sur les vecteurs de mots. Zakarya a également effectué les tests des différents modèles classiques de classification.
- **Yassine** a rédigé le rapport, en s’occupant de la présentation et de l’analyse des résultats. Yassine a également travaillé sur l’implémentation du modèle BERT en mode zero-shot, (bien que le modèle a été push par Zakarya : problème de package à ce moment, pour Yassine), en utilisant un modèle pré-entraîné pour la classification des sentiments, et a évalué les performances obtenues.

Références

- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert : Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*.
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., and Potts, C. (2011). Learning word vectors for sentiment analysis. In *Proceedings of the 49th annual meeting of the association for computational linguistics : Human language technologies*, pages 142–150.
- Mikolov, T. et al. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv :1301.3781*.
- Pang, B. and Lee, L. (2002). Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the ACL-02 conference on Empirical methods in natural language processing-Volume 10*, pages 79–86.
- Pennington, J., Socher, R., and Manning, C. D. (2014). Glove : Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543.
- Socher, R., Perelygin, A., Wu, J. Y., Chuang, J., Manning, C. D., Ng, A., and Potts, C. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.